

PIRATES LAGOON



Project Title: PIRATES LAGOON

Course: Basics Of Mechatronics

Instructor: Dr. Shady Maged

Team Members

Team 1	24P0110	Abdelrahman Hesham Mohab
	24P0056	Adham Mohamed Soliman Ahmad
	24P0014	Moaaz Ahmed Farag
	24P0095	Yassin Yahya Kelany Abdelsalam
	24P0337	Zeyad Emad AbdelMawgoud

PIRATES LAGOON

1. EXECUTIVE SUMMARY

This report presents the design, development, and implementation of Pirates Lagoon, an interactive educational toy for children aged 2-10 years. The project integrates mechanical, electrical, and software engineering principles to create a fully functional mechatronic system.

1.1 Project Overview

Pirates Lagoon is an adventure game where children insert marbles into a single entry point. The system sorts marbles by Sizes, lifts them through a vertical tower, and launches them toward corresponding target Ships. The castle-themed design includes transparent sections allowing children to observe internal mechanisms.

1.2 Technical Architecture

The system employs dual Arduino Uno microcontrollers communicating via UART protocol. Arduino 1 manages mechanical operations ; Arduino 2 handles displays, Bluetooth communication and game logic.

Three Integrated Subsystems:

Sorting Mechanism: **Servo-actuated gates direct marbles down appropriate paths.**

Lifting Mechanism: **Archimedes Screw powered by 6V DC motor lifts marbles 15cm.**

Shooting Mechanism: **Spring-loaded launcher (10mm compression) propels marbles 40cm at 45° launch angle. Projectile calculations determined required velocity of 4.22 m/s.**

1.2.1 Electrical System

Custom PCB serves as central power distribution hub. 24V 10000mA power supply. Buck converter regulates 24V to 6V for microcontrollers and DC lifting Motor.

1.2.2 User Interface

Limit switch as a shooting & reload trigger.

PIRATES LAGOON

Table of Contents

1.	EXECUTIVE SUMMARY	1
1.1	Project Overview	1
1.2	Technical Architecture.....	1
1.2.1	Electrical System	1
1.2.2	User Interface.....	1
2.	INTRODUCTION	7
2.1	Background.....	7
2.2	Project Motivation	7
2.3	Project Objectives.....	8
2.3.1	Design Objectives	8
2.3.2	Educational Objectives	9
2.3.3	Technical Objectives	9
2.4	Project Scope	10
2.5	Report Organization.....	10
3.	Product Design Specifications (PDS).....	11
3.1	Product overview:.....	11
3.2	Functional Requirements:	11
3.3	Safety requirements:	11
3.4	Target audience:.....	11
3.5	Ergonomic Toy limits:	12
3.6	Technical & Operational Specifications	12

PIRATES LAGOON

6.2	Wiring Diagrams	26
7.	Project Management	27
7.1	Work Breakdown Structure (WBS).....	27
7.2	Gantt Chart.....	28
7.3	Responsibility Assignment Matrix (RAM)	29
8.	Software Design.....	31
8.1	Arduino Firmware	31
8.2	Processing.io.....	39
9.	Safety Considerations	51

PIRATES LAGOON

List of Tables:

Table 1	17
Table 2	18
Table 3	18
Table 4	20
Table 5	20
Table 6	22
Table 7	23
Table 8	29
Table 9	30

PIRATES LAGOON

List Of Figures:

FIGURE 1	13
FIGURE 2	14
FIGURE 3	15
FIGURE 4	16
FIGURE 5	25
FIGURE 6	26
FIGURE 7	27
FIGURE 8	28

2. INTRODUCTION

2.1 Background

Children aged 2-12 spend an average of 32-56 hours weekly on screen-based entertainment, reducing hands-on learning experiences. Early STEM education significantly influences career choices, yet most STEM tools target older students, leaving a gap for children under 10.

Mechatronic toys bridge traditional play and digital engagement by combining physical mechanisms with electronic sensing and feedback. Multi-sensory learning (sight, sound, touch, motion) produces stronger retention than passive observation. Current market offerings are either too simple (limited challenge) or too expensive (beyond most budgets), with few balancing education and entertainment.

2.2 Project Motivation

This project originated from the Basics of Mechatronics course (MCT231) at Ain Shams University. Course objectives require integrating theoretical knowledge—dynamics, microcontrollers, CAD, electronics—into a comprehensive real-world design.

Our five-member team focused on creating something genuinely useful rather than merely satisfying requirements. Designing for children presents unique challenges: immediate feedback expectations, limited attention spans, rough handling, and absolute safety requirements.

The project brief requirements aligned with our goals: sorting, lifting, shooting mechanisms; dual microcontroller architecture; custom PCB; physical and digital interfaces. This provided experience with the complete product development lifecycle from user research through validation.

PIRATES LAGOON

2.3 Project Objectives

2.3.1 Design Objectives

Primary: Create integrated mechatronic system with three mechanisms (sorting, lifting, shooting) operating as closed-loop circulation with single entry point.

Mechanical:

- Design parts in Autodesk Inventor for 3D printing and laser cutting
- Calculate actuator specifications based on load/speed/torque
- Perform projectile motion analysis for 40cm $\pm$ 5cm target
- Conceal all wiring for safety
- Modular construction with standardized connectors
- Dimensions: 80×60cm maximum footprint

Electrical:

- Custom PCB for power distribution and signal routing
- Dual microcontroller architecture with reliable communication
- Safety features: free-wheeling diodes, Capacitors

Software:

- LCD display showing real-time status
- Android app with Bluetooth communication
- Error handling preventing unsafe conditions

Aesthetic:

- Castle/adventure theme appropriate for ages 2-10
- Transparent sections showing mechanisms
- Intuitive interface with large labeled buttons

PIRATES LAGOON

2.3.2 Educational Objectives

Primary: Create learning experiences emerging naturally from gameplay.

Cognitive Development:

- Cause-and-effect understanding via predictable sequences
- Pattern recognition and prediction
- Spatial reasoning through vertical/horizontal movement
- Counting and basic arithmetic via display

Scientific Literacy:

- Physics concepts: gravity, mechanical advantage, projectile motion, energy
- Engineering awareness: interconnected subsystems

Developmental Appropriateness:

- Accessible to youngest (2-4) while challenging oldest (8-10)
- Automatic operation requiring no reading
- Visual feedback compensating for limited language skills

2.3.3 Technical Objectives

Performance Targets:

- Sorting accuracy >95%
- Shooting accuracy $\pm 5\text{cm}$ for 90% of shots
- Cycle time <30 seconds

Documentation:

- Comprehensive technical report
- Engineering drawings with tolerances
- Electrical schematics and PCB layouts
- Commented source code
- Bill of Materials with suppliers
- User manual

PIRATES LAGOON

2.4 Project Scope

INCLUDED:

- User research (surveys, observations, competitive analysis)
- Design thinking methodology
- Complete 3D CAD modeling and engineering calculations
- Custom PCB design and manufacturing
- All firmware and mobile app development
- Comprehensive testing (component, subsystem, integration, system, user)
- Complete documentation (report, presentation, video, poster, Git repository)

2.5 Report Organization

Chapter 2: **User research methodology, target user analysis, design thinking process (Discover, Define, Develop, Deliver), prototyping journey.**

Chapter 3: **Product Design Specifications establishing technical requirements.**

Chapter 4: **System architecture with block diagrams, flowcharts, communication protocols.**

Chapter 5: **Mechanical design including CAD models, projectile calculations, actuator sizing.**

Chapter 6: **Electrical system with power budget, PCB design, wiring diagrams.**

Chapter 7: **Project management including WBS, Gantt chart, budget, risk analysis.**

Chapter 8: Software Design

PIRATES LAGOON

3. Product Design Specifications (PDS)

The Product Design Specifications (PDS) document serves as the technical requirements baseline for the entire project.

3.1 Product overview:

We have designed a pirates-themed interactive toy game for children aged 2–10 years. The game is designed to be engaging and educational, where the child progresses through stages instead of random play.

The child must:

- **Progress through the game story to fire the boats.**
- **Plan his game and focus on shots management to reach the highest score.**

3.2 Functional Requirements:

1. Sound effects
2. On/Off switch
3. Lights and screen displays

3.3 Safety requirements:

1. Hidden cables and fully safe electrical organization
2. Closed frame covering all the game contents
3. No sharp edges
4. Materials are BPA-free and non-toxic

3.4 Target audience:

Primary:

- **Children aged 2–10 years**

Secondary:

- **Parents looking for educational and skill-developing toys**
- **Arcade centers**

PIRATES LAGOON

3.5 Ergonomic Toy limits:

1. Button diameter ≥ 18 mm
2. Button activation force = 1–2.5 N
3. Grip circumference = 90–130 mm
4. Shooting guns moving range = 120–150 degrees
5. Bullets speed = 4–5 m/s
6. Control response time ≤ 0.3 s
7. Toy weight < 3 kg
8. $\leq 1\text{m} \times 1\text{m}$ plate that carries the whole game
9. Two bullets ball-shaped of radii 1 and 1.75 cm

3.6 Technical & Operational Specifications

3.6.1 Power Consumption:

Main Input: 24V DC via a 10A regulated power supply.

System Peak Load: ~105W (Approx. 5.4A during simultaneous solenoid firing and motor movement).

Efficiency: Regulated via DC-DC Buck Converters to minimize heat loss and maximize component life.

3.6.2 Product Lifespan:

Mechanical Endurance: The shooting and lifting mechanisms are designed for >50,000 cycles.

Electronic MTBF: Control systems (Arduinos and peripherals) are rated for 10,000 operational hours.

Maintenance Interval: Expected sensor calibration and mechanical lubrication every 1,000 hours of play.

Duty Cycle Protection: Software-limited firing rate ensures the 24V "overcharged" solenoid does not exceed thermal limits, preserving coil integrity for the life of the product.

PIRATES LAGOON

4. System Architecture

4.1 System Overview

The system consists of three main subsystems integrated through a central control architecture with dual microcontroller configuration.

4.2 System Flowchart

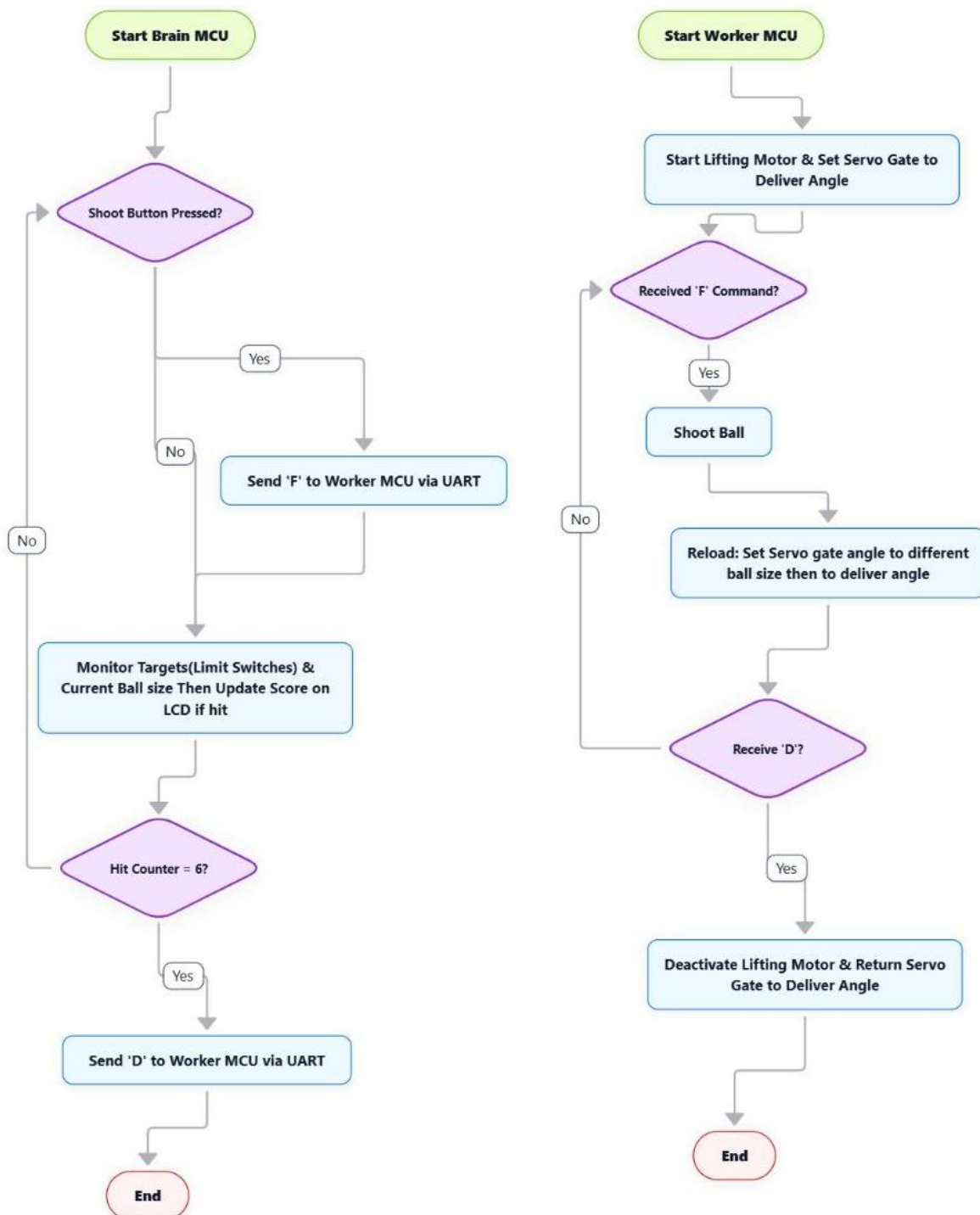


FIGURE 1

PIRATES LAGOON

4.3 System Block Diagram

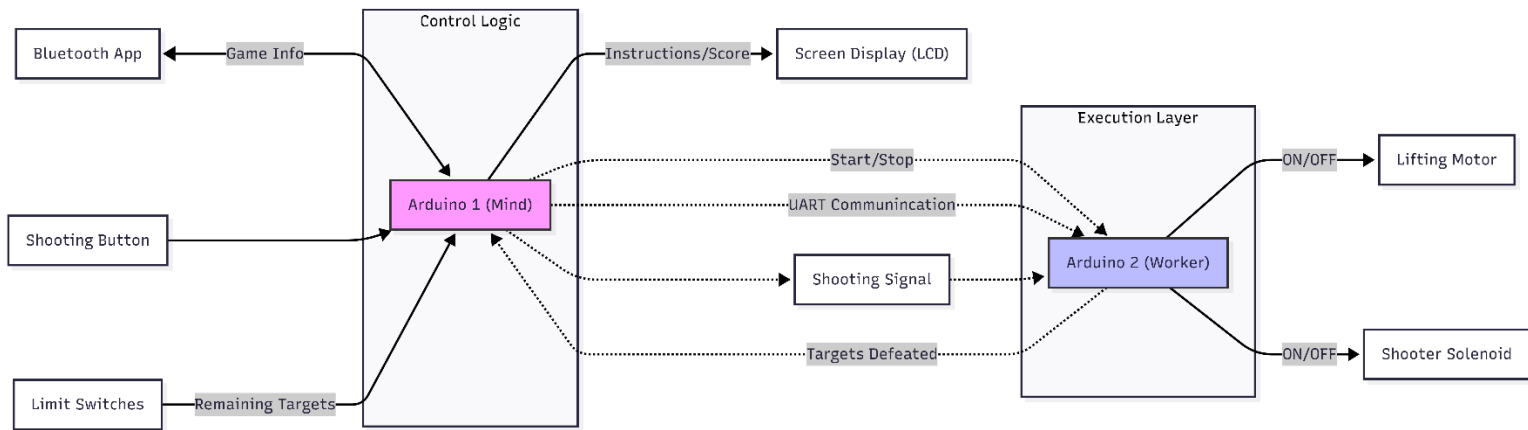


FIGURE 2

PIRATES LAGOON

5. Mechanical Design

5.1 CAD Design Overview

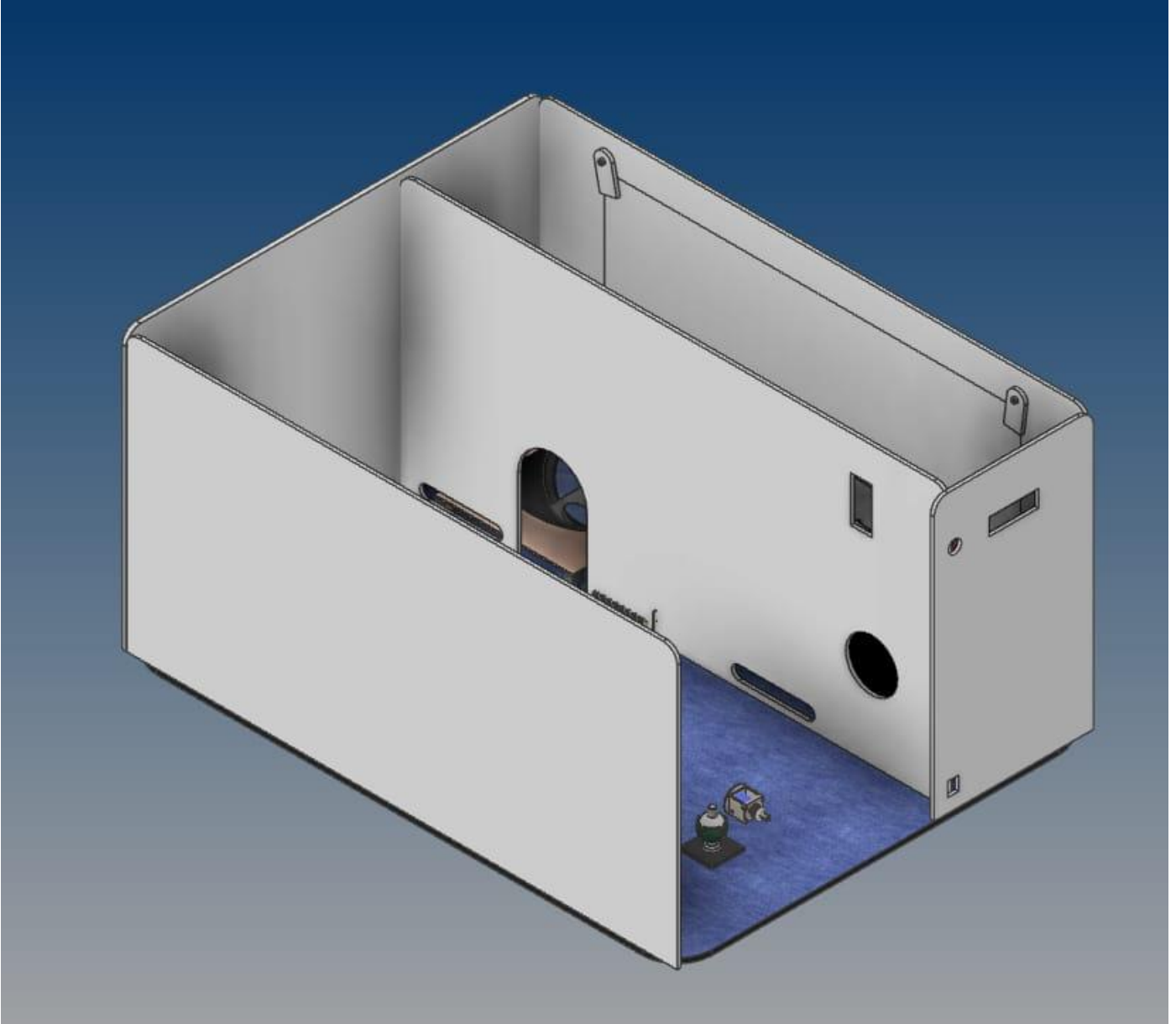


FIGURE 3

PIRATES LAGOON

5.4 Shooting Mechanism

5.4.1 Projectile Motion Calculations

Parameter	Value
Launch angle	46 deg
Horizontal target dist.	1.1 m
Target height (ground)	0.6 m
Shooter height	0.15 m
y relative to launch	0.6 - 0.15 = 0.45 m
Ball mass 1	2.0 g
Ball mass 2	6.15 g
Gravity	9.81 m/s²

Table 1

5.4.1.1 Problem Setup and Given Values

mounted 150mm above the ground, so the launch point height $h = 0.15\text{m}$. This means the trajectory y is measured relative to the launch point:

5.4.1.2 Projectile Equation and Solving for v_0

Standard projectile equation (y measured from launch point):

$$y = x \tan(\theta) - g x^2 / (2 v_0^2 \cos^2(\theta))$$

Rearranging to solve for v_0^2 :

$$v_0^2 = g x^2 / [2 \cos^2(\theta) (x \tan(\theta) - y_{\text{rel}})]$$

Substituting values:

$$\text{Numerator} = g x^2 = 9.81 \times 1.1^2 = 11.870 \times \tan(\theta) = 1.1 \times \tan(46) = 1.1 \times 1.0355 = 1.1391 \times \tan(\theta) - y = 1.1391 - 0.45 = 0.6891$$

$$2 \cos^2(\theta) = 2 \times \cos^2(46) = 2 \times 0.4823 = 0.9646$$

$$\text{Denominator} = 0.9646 \times 0.6891 = 0.6650$$

$$v_0^2 = 11.870 / 0.6650 = 17.85 \text{ m}^2/\text{s}^2$$

$$v_0 = \sqrt{17.85} = 4.22 \text{ m/s}$$

PIRATES LAGOON

5.4.1.3 Kinetic Energy for Each Ball

Kinetic energy at launch: $E = (1/2) m v_0^2$

Ball 1 ($m_1 = 2\text{g} = 0.002\text{ kg}$):

$$E_1 = (1/2) \times 0.002 \times 4.22^2 = 0.001 \times 17.85 = 17.85\text{ mJ}$$

Ball 2 ($m_2 = 6.15\text{g} = 0.00615\text{ kg}$):

$$E_2 = (1/2) \times 0.00615 \times 4.22^2 = 0.003075 \times 17.85 = 54.88\text{ mJ}$$

Apply safety factor = 1.4 (to account for air resistance, estimation errors, real-world losses):

$$E_{1_safe} = 17.85 \times 1.4 = 24.99\text{ mJ} \sim 25\text{ mJ}$$

$$E_{2_safe} = 54.88 \times 1.4 = 76.84\text{ mJ} \sim 77\text{ mJ}$$

Ball	Mass	E_kinetic	x SF 1.4	E_safe (design value)
Ball 1	2 g	17.85 mJ	x 1.4	25 mJ
Ball 2	6.15 g	54.88 mJ	x 1.4	77 mJ
Worst case	6.15 g	--	--	77 mJ (use this for sizing)

Table 2

5.4.1.4 Application Requirements

Requirement	Value
Function	Launch ball (push)
Required energy	$\geq 77\text{ mJ}$
Max stroke available	10 mm
Supply voltage	24 V DC
Duty cycle	User Triggered
Worst case ball mass	6.15 g

Table 3

PIRATES LAGOON

5.4.1.5 Calculate Inertia of Moving Part

The solenoid is a LINEAR actuator. The moving parts are the plunger (steel rod) and the ball. For linear systems, the relevant quantity is MASS (not rotational J), used in $F = m \times a$ and $W = (1/2) m v^2$.

Plunger (solid steel cylinder)

Geometry: solid cylinder --> $\rho_{\text{steel}} = 7800 \text{ kg/m}^3$

From JF-0826B spec: $D_{\text{plunger}} = 7.4 \text{ mm}$ --> $r = 3.7 \text{ mm} = 0.0037 \text{ m}$

Estimated length $L = 40 \text{ mm} = 0.040 \text{ m}$ (from total length 57mm - frame 26mm + stroke 10mm)

Volume = $\pi \times r^2 \times L = \pi \times (0.0037)^2 \times 0.040 = 1.72 \times 10^{-6} \text{ m}^3$

$m_{\text{plunger}} = \rho \times V = 7800 \times 1.72 \times 10^{-6} = 13.42 \text{ g} = 0.01342 \text{ kg}$

Ball (solid sphere, worst case)

$m_{\text{ball}} = 6.15 \text{ g} = 0.00615 \text{ kg}$ (heaviest ball -- critical case)

$J_{\text{cm}} = (2/5) m r^2 = (2/5) \times 0.00615 \times (0.0175)^2 = 7.53 \times 10^{-7} \text{ kg.m}^2$

5.4.1.6 Required Force

The solenoid accelerates the plunger + ball system over stroke $s = 10 \text{ mm}$. At end of stroke, the plunger stops and transfers velocity to the ball. This is a two-phase problem:

Phase 1 -- Acceleration over stroke (F acts on both plunger + ball):

2 Work-energy: $F \times s = (1/2) \times m_{\text{eff}} \times v_{\text{contact}}^2$ $v_{\text{contact}} = \sqrt{2 \times F \times s / m_{\text{eff}}}$

Phase 2 -- Elastic collision (plunger stops, ball launches):

By conservation of momentum (elastic collision):

$v_{\text{ball}} = [2 \times m_{\text{plunger}} / (m_{\text{plunger}} + m_{\text{ball}})] \times v_{\text{contact}}$

Setting $v_{\text{ball}} \geq v_0 = 4.22 \text{ m/s}$ and solving for F_{min} :

$x = v_0 \times (m_{\text{plunger}} + m_{\text{ball}}) / (2 \times m_{\text{plunger}}) \times = 4.22 \times (0.01342 + 0.00615) / (2 \times 0.01342) = 3.079 \text{ m/s}$

$F_{\text{min}} = x^2 \times m_{\text{eff}} / (2 \times s)$

$F_{\text{min}} = (3.079)^2 \times 0.01957 / (2 \times 0.010) = 9.26 \text{ N}$

Apply safety factor $SF = 1.5$:

$F_{\text{required}} = 9.26 \times 1.5 = 13.9 \text{ N}$

Requirement	Value needed	JF-0530B	JF-0826B
Force	$\geq 13.9 \text{ N}$	5 N	20 N
Stroke	$\geq 10 \text{ mm}$	10 mm	10 mm
Voltage	24 V DC for Overcharging	12 V DC	12 V DC
Current	Any	300 mA	2 A

PIRATES LAGOON

Table 4

JF-0530B fails the force requirement (only 5N vs 11.6N needed). JF-0826B provides 20N with 1.74x margin. Selected: JF-0826B.

5.5 Lifting Mechanism

5.5.1 Actuator Sizing Calculations

The lifting motor raises balls from the magazine/storage area up to the shooter (height = 150mm). The mechanism uses a drum or spool to wind a cord.

Requirement	Value
Function	Lift balls to shooter height
Lift height	150 mm = 0.15 m
Load (worst case)	10 balls = 61.5 g
Speed requirement	Slow controlled
Self-locking	Yes preferred
Supply voltage	6 V DC
Drum radius assumed	10 mm

Table 5

5.5.1.1 Inertia of All Rotating Parts

Auger (Archimedes screw) --> solid cylinder approximation

$$m_{\text{auger}} = 120 \text{ g} = 0.120 \text{ kg} \mid D = 100 \text{ mm} \rightarrow r = 50 \text{ mm} = 0.050 \text{ m} \mid L = 250 \text{ mm}$$

$$J_{\text{auger}} = (1/2) \times m \times r^2 = (1/2) \times 0.120 \times (0.050)^2$$

$$J_{\text{auger}} = (1/2) \times 0.120 \times 0.0025 = 1.50 \times 10^{-4} \text{ kg.m}^2$$

Ball inside auger flight (worst case: heaviest ball)

The ball rides inside the auger flight at approximately half the auger radius from axis.

$$m_{\text{ball}} = 6.15 \text{ g} = 0.00615 \text{ kg} \mid D_{\text{ball}} = 35 \text{ mm} \rightarrow r_{\text{ball}} = 17.5 \text{ mm} = 0.0175 \text{ m} \mid d \text{ (ball center from rotation axis)} = r_{\text{auger}} / 2 = 25 \text{ mm} = 0.025 \text{ m}$$

$$\text{Solid sphere: } J_{\text{cm}} = (2/5) \times m \times r^2$$

$$J_{\text{cm}} = (2/5) \times 0.00615 \times (0.0175)^2 = 7.53 \times 10^{-7} \text{ kg.m}^2$$

$$\text{Parallel axis theorem: } J_{\text{axis}} = J_{\text{cm}} + m \times d^2$$

$$J_{\text{axis}} = 7.53 \times 10^{-7} + 0.00615 \times (0.025)^2$$

$$J_{\text{axis}} = 7.53 \times 10^{-7} + 3.844 \times 10^{-6} = 4.60 \times 10^{-6} \text{ kg.m}^2$$

PIRATES LAGOON

Total Load Inertia:

$$J_L = J_{\text{auger}} + J_{\text{ball}}$$

$$J_L = 1.50 \times 10^{-4} + 4.60 \times 10^{-6} = 1.546 \times 10^{-4} \text{ kg.m}^2$$

5.5.1.2 Motor Inertia

$$J_m / J_L = 1/1 \text{ to } 1/10 \rightarrow \text{use worst case } J_m = J_L$$

$$J_m = 1.546 \times 10^{-4} \text{ kg.m}^2$$

$$J_{eq} = J_L + J_m = 1.546 \times 10^{-4} + 1.546 \times 10^{-4}$$

$$J_{eq} = 3.092 \times 10^{-4} \text{ kg.m}^2$$

5.5.1.3 Motion Profile and Angular Acceleration

symmetric 1/3-1/3-1/3, $t_{acc} = 1.0 \text{ s}$ (slow ramp for lifter):

$$\omega_{max} = 2 \times \pi \times 30 / 60 = 3.14 \text{ rad/s}^2 \text{ (motor output = 30 RPM)}$$

$$2 \theta_{acc} = \omega_{max} / t_{acc} = 3.14 / 1.0 = 3.14 \text{ rad/s } \theta_{run} = 0 \text{ (constant speed)}$$

$$2 \theta_{dec} = -3.14 \text{ rad/s}^2$$

5.5.1.4 Resisting Torque

$$T_R = T_{\text{damping}} + T_{\text{friction}} + T_L(\text{eff})$$

$T_L(\text{eff})$ -- gravity load on ball being lifted along auger axis:

$$F_{\text{gravity}} = m_{\text{ball}} \times g = 0.00615 \times 9.81 = 0.0603 \text{ N}$$

$$-3 T_{\text{gravity}} = F_{\text{gravity}} \times r_{\text{effective}} = 0.0603 \times 0.025 = 1.51 \times 10^{-3} \text{ N.m}$$

T_{friction} -- shaft bearing friction:

$$T_{\text{fric}} = \mu \times m_{\text{rotating}} \times g \times r_{\text{shaft}}$$

$$T_{\text{fric}} = 0.15 \times 0.12615 \times 9.81 \times 0.0025 = 4.64 \times 10^{-4} \text{ N.m (small)}$$

$$T_R = T_{\text{gravity}} + T_{\text{fric}} = 1.51 \times 10^{-3} + 4.64 \times 10^{-4} = 1.97 \times 10^{-3} \text{ N.m}$$

5.5.1.5 Motor Torque T_m per Phase

Acceleration:

$$T_{acc} = J_{eq} \times \theta_{acc} + T_R = 3.092 \times 10^{-4} \times 3.14 + 1.97 \times 10^{-3}$$

$$T_{acc} = 9.71 \times 10^{-4} + 1.97 \times 10^{-3} = 2.94 \times 10^{-3} \text{ N.m}$$

Constant speed phase:

$$T_{run} = 0 + T_R = 1.97 \times 10^{-3} \text{ N.m}$$

Deceleration phase:

$$T_{dec} = 3.092 \times 10^{-4} \times (-3.14) + 1.97 \times 10^{-3} = 1.00 \times 10^{-3} \text{ N.m}$$

$$T_{peak} = \max(T_{acc}, T_{run}, T_{dec}) = T_{acc} = 2.94 \times 10^{-3} \text{ N.m}$$

T_{rms} formula :

$$T_{rms} = \sqrt{(T_{acc}^2 \times t_{acc} + T_{run}^2 \times t_{run} + T_{dec}^2 \times t_{dec}) / (t_{acc} + t_{run} + t_{dec})}$$

$$T_{rms} = \sqrt{((2.94 \times 10^{-3})^2 \times 1 + (1.97 \times 10^{-3})^2 \times 1 + (1.00 \times 10^{-3})^2 \times 1) / 3}$$

PIRATES LAGOON

$$T_{rms} = \sqrt{(8.64 \times 10^{-6} + 3.88 \times 10^{-6} + 1.00 \times 10^{-6}) / 3}$$

$$T_{rms} = \sqrt{4.51 \times 10^{-6}} = 2.12 \times 10^{-3} \text{ N.m}$$

With SF = 1.5 applied to T_{peak} :

5.5.1.6 Motor Selection

$$T_{required} = T_{peak} \times SF = 2.94 \times 10^{-3} \times 1.5 = 4.41 \times 10^{-3} \text{ N.m} = 0.045 \text{ kg.cm}$$

Parameter	Required	N20 6V 30RPM 1:1000	Status
T_{rated} (cont.)	$\geq T_{rms} =$ 2.12 mNm	1.2 kg.cm = 117.7 mNm	PASS (55x)
T_{peak}	$\geq 4.41 \text{ mNm}$	1.2 kg.cm = 117.7 mNm	PASS (26x)
Output speed	30 RPM	30 RPM	PASS
Voltage	6 V DC	6 V DC	PASS

Table 6

5.6 Sorting Mechanism

5.6.1 Module Description and Motor Type

The motor rotates a gate without gear ratio to a determined positions according to the design in range of 180 degree to move the balls from 2 places to 1 place. So, we choose that the best type of motors to carry this function is (servo position motor)

5.6.2 Inertia of Each Part (Parallel Axis Theorem)

A) Small Ball (m = 2g, D = 20mm, d_{axis} = 38.5mm)

Geometry: solid sphere --> $J_{cm} = (2/5) m r^2$

$$J_{cm} = (2/5) \times 0.002 \times (0.01)^2 = 8.0 \times 10^{-8} \text{ kg.m}^2$$

$$\text{Parallel axis: } J_{axis} = J_{cm} + m \times d^2 = 8 \times 10^{-8} + 0.002 \times (0.0385)^2$$

$$J_{small} = 8 \times 10^{-8} + 2.965 \times 10^{-6} = 3.04 \times 10^{-6} \text{ kg.m}^2$$

B) Big Ball (m = 6.15g, D = 35mm, d_{axis} = 30mm)

$$J_{cm} = (2/5) \times 0.00615 \times (0.0175)^2 = 7.53 \times 10^{-7} \text{ kg.m}^2$$

$$J_{axis} = 7.53 \times 10^{-7} + 0.00615 \times (0.030)^2 = 7.53 \times 10^{-7} + 5.535 \times 10^{-6}$$

$$J_{big} = 6.29 \times 10^{-6} \text{ kg.m}^2 (\sim 6.3 \times 10^{-6})$$

C) Shaft (m = 1.4g, D = 5mm)

Neglected -- due to very low mass ($m \ll D$), J_{shaft} is negligible.

PIRATES LAGOON

D) Diffuser Head (Disk) (m = 20g, R = 100mm)

Geometry: equivalent disk (with 2 half-holes -- real J slightly less than estimate)

$$J_{\text{disk}} = (1/2) \times m \times R^2 = 0.5 \times 0.020 \times (0.100)^2 = 1.0 \times 10^{-4} \text{ kg.m}^2$$

E) Nut M12 (m = 15g, r_equiv = 6mm)

Geometry: equivalent disk (hex nut approximated as disk)

$$J_{\text{nut}} = (1/2) \times m \times r^2 = 0.5 \times 0.015 \times (0.006)^2 = 2.7 \times 10^{-7} \text{ kg.m}^2$$

Total Inertia:

$$J_{\text{total}} = J_{\text{small}} + J_{\text{big}} + J_{\text{disk}} + J_{\text{nut}}$$

$$J_{\text{total}} = 3.04 \times 10^{-6} + 6.3 \times 10^{-6} + 1.0 \times 10^{-4} + 2.7 \times 10^{-7}$$

$$J_{\text{total}} = 1.096 \times 10^{-4} \text{ kg.m}^2$$

Part	Mass	Geometry	J (kg.m ²)
A - Small ball	2 g	Solid sphere	3.04×10^{-6}
B - Big ball	6.15 g	Solid sphere	6.30×10^{-6}
C - Shaft	1.4 g	Cylinder	NEGLECTED
D - Diffuser	20 g	Disk R=100mm	1.00×10^{-4}
E - Nut M12	15 g	Equiv. disk	2.70×10^{-7}
TOTAL	44.55 g	--	1.096×10^{-4}

Table 7

5.6.3 Torque Calculation ($T = J \times \alpha + T_{\text{fric}} + T_{\text{load}}$)

Angular acceleration:

Assume servo accelerates from 0 to ω_{max} in $t_{\text{acc}} = 0.5 \text{ s}$

$$\omega_{\text{max}} = 2 \times \pi = 6.28 \text{ rad/s (approx. 60 RPM -- typical servo speed)}$$

$$2 \alpha = \omega_{\text{max}} / t_{\text{acc}} = 6.28 / 0.5 = 12.56 \text{ rad/s}^2$$

Inertia torque:

$$T_{\text{inertia}} = J_{\text{total}} \times \alpha = 1.096 \times 10^{-4} \times 12.56 = 1.377 \times 10^{-3} \text{ N.m}$$

Load torque:

$T_{\text{load}} = 0$ (horizontal rotation -- gravity acts perpendicular to motion, does no work)

Friction torque (at shaft bearing):

$$T_{\text{fric}} = \mu \times N_{\text{bearing}} \times r_{\text{shaft}}$$

$$\mu = 0.35 \text{ (dry contact, plastic/metal bearing)} \quad N_{\text{bearing}} = m_{\text{rotating}} \times g = 0.015 \times 9.81 =$$

$$0.437 \text{ N} \quad r_{\text{shaft}} = D_{\text{shaft}}/2 = 5/2 = 2.5 \text{ mm} = 0.0025 \text{ m}$$

$$T_{\text{fric}} = 0.052 \text{ N.m}$$

Total required torque (with safety factor SF = 1.5):

$$T_{\text{total}} \text{ (before SF)} = T_{\text{inertia}} + T_{\text{fric}} + T_{\text{load}}$$

$$T_{\text{required}} = T_{\text{total}} \times \text{SF} = 0.05338 \times 1.8 = 0.096084 \text{ N.m}$$

PIRATES LAGOON

5.6.4 Servo Motor Selection

The SG90 is selected -- it provides 38x torque margin over the requirement, is compact, lightweight (9g), inexpensive, and directly controllable by Arduino PWM.

- **Weight:** 9g
- **Rotation:** 180 degrees (standard)
- **Torque:** 1.8 kg/cm (at 4.8V)
- **Speed:** 0.12 seconds/60° (no load, 4.8V)
- **Gears:** Plastic
- **Connections:** Brown (GND), Red (5V), Orange (Signal)

6. Electrical Design

6.1 Custom PCB Design

6.1.1 PCB schematic

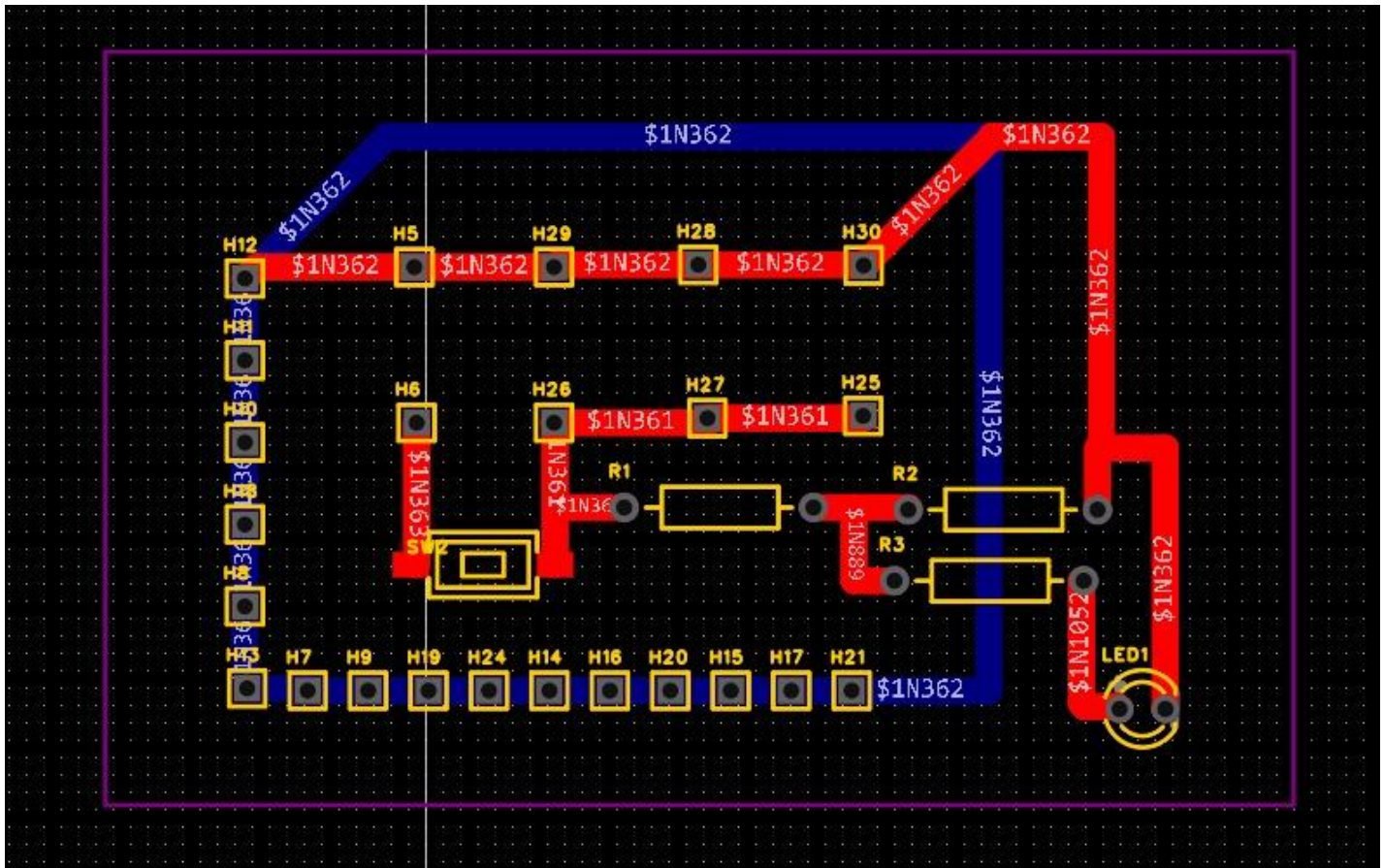


FIGURE 5

7. Project Management

7.1 Work Breakdown Structure (WBS)

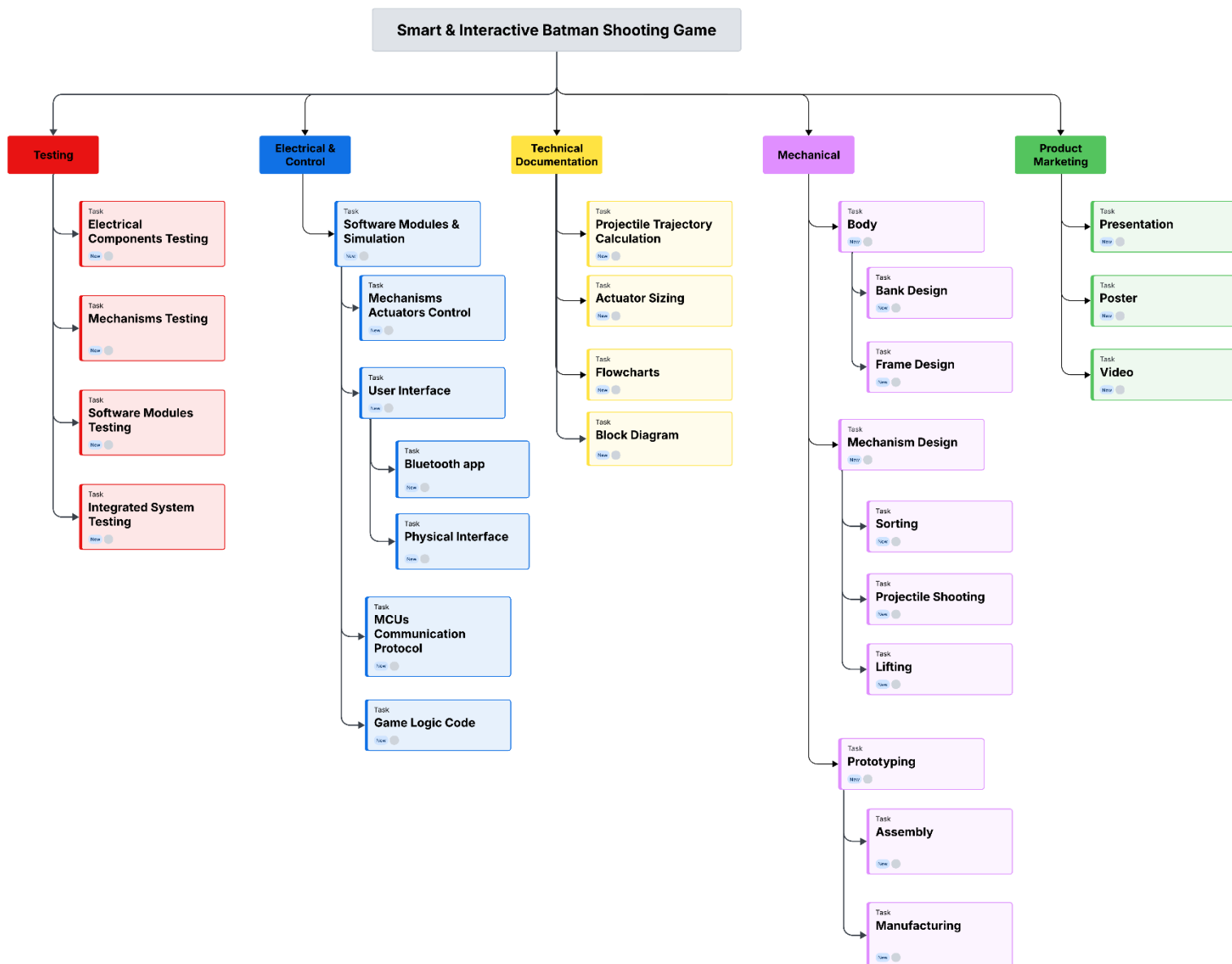


FIGURE 7

PIRATES LAGOON

7.2 Gantt Chart

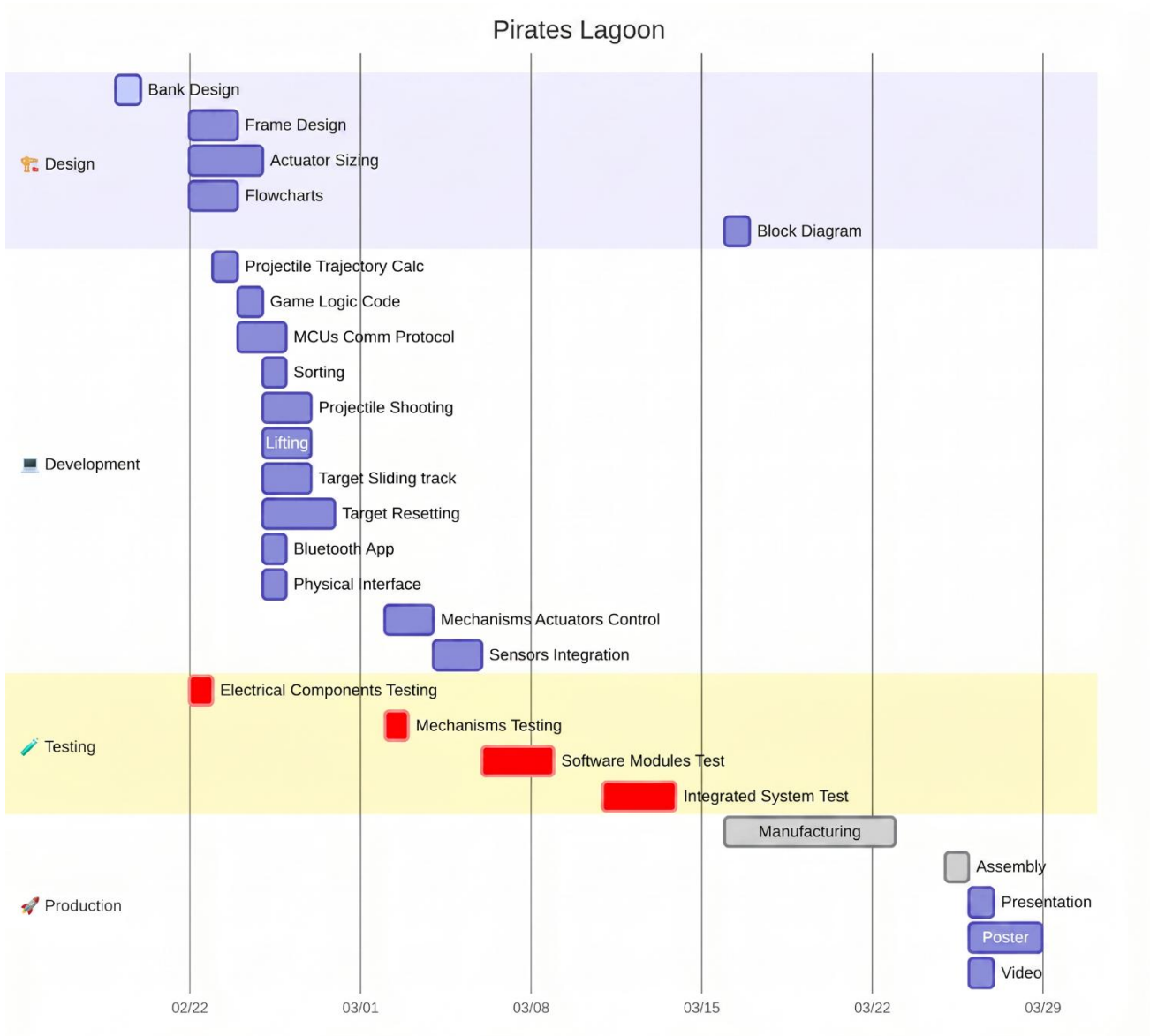


FIGURE 8

PIRATES LAGOON

7.3 Responsibility Assignment Matrix (RAM)

RESPONSIBILITY ASSIGNMENT MATRIX (RAM)							
Smart & Interactive Batman Shooting Game RACI: R=Responsible A=Accountable C=Consulted I=Informed							
Category	ID	Task / Deliverable	Abdelrahman	Zeyad	Yassin	Adham	Moaaz
Testing	T1	Electrical Components Testing	R	C	I	C	A
	T2	Mechanisms Testing	C	I	R	A	C
	T3	Software Modules Testing	A	R	C	I	C
	T4	Integrated System Testing	C	A	C	R	C
Electrical & Control	E1	Software Modules & Simulation	R	A	C	I	I
	E2	Mechanisms Actuators Control	A	R	I	C	I
	E3	User Interface	R	A	C	I	C
	E4	Bluetooth App	A	R	I	C	I
	E5	Physical Interface	R	A	C	I	I
	E6	MCUs Communication Protocol	A	R	C	I	I
	E7	Game Logic Code	R	A	I	C	C
Technical Documentation	D1	Projectile Trajectory Calculation	C	I	I	C	R
	D2	Actuator Sizing	I	A	I	I	R
	D3	Flowcharts	A	R	I	I	C
	D4	Block Diagram	C	I	C	A	R
Mechanical	M1	Body – Bank Design	C	I	R	A	C
	M2	Body – Frame Design	I	C	A	R	R
	M3	Mechanism Design – Sorting	C	I	R	A	I
	M4	Mechanism Design – Projectile Shooting	I	C	A	R	I
	M5	Mechanism Design – Lifting	C	I	R	A	I
	M6	Prototyping – Assembly	I	C	A	R	C
	M7	Prototyping – Manufacturing	C	I	R	A	I
Product Marketing	P1	Presentation	C	I	I	C	R
	P2	Poster	I	C	C	I	R
	P3	Video	A	C	I	C	R

LEGEND		
R	Responsible	Does the actual work on this task
A	Accountable	Ultimately answerable; approves/reviews output
C	Consulted	Input sought; two-way communication
I	Informed	Kept updated; one-way communication

Table 8

PIRATES LAGOON

TEAM CONTRIBUTION ANALYSIS

All formulas linked live to RAM sheet | Weighted score: $R=3 \text{ pts} \cdot A=2 \text{ pts} \cdot C=1 \text{ pt} \cdot I=0 \text{ pts}$

A. RACI Role Counts per Person						
Role	Abdelrahman	Zeyad	Yassin	Adham	Moaaz	Total
R – Responsible	5	5	5	4	7	26
A – Accountable	6	6	3	6	1	22
C – Consulted	9	6	8	7	8	38
I – Informed	5	8	9	8	9	39
TOTAL assignments	25	25	25	25	25	125

B. Weighted Contribution % per Person						
Metric	Abdelrahman	Zeyad	Yassin	Adham	Moaaz	Team
Weighted Score ($R \times 3 + A \times 2 + C \times 1$)	36	33	29	31	31	160
% of Team Weighted Score	22.5%	20.6%	18.1%	19.4%	19.4%	100.0%

C. Responsible (R) Tasks per Person by Category						
Category	Abdelrahman	Zeyad	Yassin	Adham	Moaaz	Total
Testing	1	1	1	1	0	4
Electrical & Control	4	3	0	0	0	7
Technical Documentation	0	1	0	0	3	4
Mechanical	0	0	4	3	1	8
Product Marketing	0	0	0	0	3	3
GRAND TOTAL (R)	5	5	5	4	7	26

Table 9

PIRATES LAGOON

8. Software Design

8.1 Arduino Firmware

Mind

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <DFRobotDFPlayerMini.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial dfSerial(11, 9); // RX=D11, TX=D9
DFRobotDFPlayerMini dfPlayer;

#define SHOOT_BUTTON 4

const char* playerNames[] = {"SHOKRY", "ZEYAD", "MOAZ", "YASSIN", "ADHAM"};
int trackBase[] = {1, 8, 15, 22, 29};
int workingPlayers[] = {0, 1, 2, 3}; // ADHAM removed from active track logic

// Simple state tracking flags (No enum state machines)
bool isGameActive = false;
bool isGameWon = false;
bool isGameLost = false;

int allyIndex = -1;
bool targetsHit[5] = {false, false, false, false, false};
int targetsHitCount = 0;
int totalScore = 0;
bool waitingForHit = false;
bool ballIsBig = true;
unsigned long endScreenTime = 0;

void playTrack(int trackNum) {
    dfSerial.listen();
    dfPlayer.playMp3Folder(trackNum);
}

void updateLCD() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ally:");
    lcd.print(playerNames[allyIndex]);
    lcd.setCursor(0, 1);
    lcd.print("Sc:");
    lcd.print(totalScore);
}
```


PIRATES LAGOON

```

    lcd.print(" ");
    lcd.print(targetsHitCount);
    lcd.print("/4 ");
    lcd.print(ballIsBig ? "[BIG]" : "[SML]");
}

void initGame() {
    for (int i = 0; i < 5; i++) targetsHit[i] = false;
    targetsHitCount = 0;
    totalScore      = 0;
    waitingForHit    = false;
    ballIsBig        = true;
    isGameActive     = true;
    isGameWon        = false;
    isGameLost       = false;

    // Single-byte Reset token to Worker
    Serial.write('R');
    Serial.flush();
    while (Serial.available()) Serial.read();

    randomSeed(analogRead(A0));
    allyIndex = random(0, 5);

    // Sync token sequence to Processing.io over Bluetooth
    Serial.print('I');
    Serial.print(allyIndex + 1); // Processing expects 1-5 format
    Serial.print('\n');
    Serial.flush();

    playTrack(trackBase[allyIndex]);

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Ally:");
    lcd.print(playerNames[allyIndex]);
    lcd.setCursor(0, 1);
    lcd.print("Sc:0  0/4 [BIG]");
}

void setup() {
    pinMode(SHOOT_BUTTON, INPUT_PULLUP);

    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.print("Starting...");
}

```

PIRATES LAGOON

```

delay(2000);
dfSerial.begin(9600);
delay(1000);

dfPlayer.begin(dfSerial);
delay(500);
dfPlayer.volume(25);
dfPlayer.EQ(DFPLAYER_EQ_NORMAL);

lcd.clear();
lcd.print("DFPlayer Ready");
delay(1000);

Serial.begin(9600); // Bluetooth Link & Worker Shared Bus
delay(500);
while (Serial.available()) Serial.read();

initGame();
}

void loop() {
  // Post-game auto-restart delay tracking loop
  if (!isGameActive && (millis() - endScreenTime >= 5000)) {
    initGame();
    return;
  }

  // Handle playing state updates
  if (isGameActive) {
    // Firing Request Handler
    if (digitalRead(SHOOT_BUTTON) == LOW && !waitingForHit) {
      Serial.write('F'); // Single-byte Fire token down hard wire to Worker
      Serial.flush();
      waitingForHit = true;

      // Pass token out to sync the Processing screen animation update
      Serial.print("FIRE\n");
      Serial.flush();

      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("  ** FIRE! ** ");
      lcd.setCursor(0, 1);
      lcd.print(ballIsBig ? "Ball:BIG  +1pt" : "Ball:SML  +2pt");

      while (digitalRead(SHOOT_BUTTON) == LOW);
      delay(200); // Simple mechanical button debounce window
    }
  }
}

```

PIRATES LAGOON

```
// Inbound evaluation parsing from Worker via RX line
if (Serial.available() && waitingForHit) {
    char hit = Serial.read();
    int irIndex = hit - '1'; // Converts '1'-'5' array bounds back to 0-4

    if (irIndex >= 0 && irIndex <= 4) {
        waitingForHit = false;

        bool thisBallBig = ballIsBig;
        ballIsBig = !ballIsBig;

        // Loss Condition Check: Friend Target Tripped
        if (irIndex == allyIndex) {
            isGameActive = false;
            isGameLost = true;

            playTrack(trackBase[allyIndex] + 5); // Play lose track

            Serial.print("LOSE\n"); // Notify Processing over Bluetooth
            Serial.flush();

            lcd.clear();
            lcd.setCursor(0, 0);
            lcd.print(" ** GAME OVER **");
            lcd.setCursor(0, 1);
            lcd.print("Hit Friend Ship!");
            endScreenTime = millis();
            return;
        }
    }

    // Score Calculation & Hit Registry Updates
    if (!targetsHit[irIndex]) {
        targetsHit[irIndex] = true;
        targetsHitCount++;

        int points = thisBallBig ? 1 : 2;
        totalScore += points;

        // Notify Processing over Bluetooth to sink this specific ship item
        Serial.print('H');
        Serial.print(irIndex + 1); // Match 1-5 alignment metrics
        Serial.print('\n');
        Serial.flush();

        int targetNum = 0;
        for (int i = 0; i < 4; i++) {
            if (workingPlayers[i] == allyIndex) continue;

```

PIRATES LAGOON

```

        targetNum++;
        if (workingPlayers[i] == irIndex) break;
    }

    playTrack(trackBase[allyIndex] + targetNum);

    // Win Evaluation Loop Validation
    if (targetsHitCount >= 4 && totalScore >= 6) {
        isGameActive = false;
        isGameWon = true;

        playTrack(trackBase[allyIndex] + 6); // Play win track

        Serial.print("WIN\n");
        Serial.flush();

        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("  ** YOU WIN ** ");
        lcd.setCursor(0, 1);
        lcd.print("Score:");
        lcd.print(totalScore);
        endScreenTime = millis();
        return;
    }
}
updateLCD();
}
}
}
}
}

```

PIRATES LAGOON

Worker

```
#include <Servo.h>

// Even pattern hardware pin layouts
const int IR_PINS[5] = {4, 6, 8, 10, 12};
const int RELAY_SOLENOID_PIN = 2;
const int SERVO_MOTOR_PIN = 9;

Servo sortingServo;

// Tracking arrays for edge-trigger filtering (Active-Low IR Sensors)
bool lastSensorState[5] = {HIGH, HIGH, HIGH, HIGH, HIGH};

// Solenoid Pulse Timers
unsigned long solenoidTriggerTime = 0;
const unsigned long SOLENOID_PULSE_DURATION = 120; // 120ms power cycle
bool isSolenoidActive = false;

// Mechatronic Reload Timeline (Completely flat alternative to state machines)
unsigned long reloadStartTime = 0;
const unsigned long SERVO_PUSH_DURATION = 300; // Arm sweep out time window
const unsigned long SERVO_RETURN_DURATION = 600; // Total arm timeline back home
bool isReloading = false;

void setup() {
    Serial.begin(9600); // Direct hardware UART cross link to Mind Uno

    pinMode(RELAY_SOLENOID_PIN, OUTPUT);
    digitalWrite(RELAY_SOLENOID_PIN, HIGH); // Force HIGH (Active-Low relay safe off-
state)

    for (int i = 0; i < 5; i++) {
        pinMode(IR_PINS[i], INPUT_PULLUP);
    }

    sortingServo.attach(SERVO_MOTOR_PIN);
    sortingServo.write(0); // Set mechanism home gate locked
}

void loop() {
    unsigned long currentMillis = millis();

    // 1. SOLENOID BURST CUTOFF SAFETY TIMELINE
    if (isSolenoidActive && (currentMillis - solenoidTriggerTime >=
SOLENOID_PULSE_DURATION)) {
        digitalWrite(RELAY_SOLENOID_PIN, HIGH); // Force relay off safely
```

PIRATES LAGOON

```

isSolenoidActive = false;

// Hand down loading instructions to servo timeline instantly
sortingServo.write(45); // Open track gate chute
reloadStartTime = currentMillis;
isReloading = true;
}

// 2. FLAT MECHATRONIC RELOAD TIMELINE HANDLING
if (isReloading) {
    unsigned long elapsedTime = currentMillis - reloadStartTime;

    if (elapsedTime >= SERVO_PUSH_DURATION && elapsedTime < SERVO_RETURN_DURATION)
    {
        sortingServo.write(0); // Sweep chamber track gate block back home
    }
    else if (elapsedTime >= SERVO_RETURN_DURATION) {
        isReloading = false; // Reset cycle flag completely
    }
}

// 3. PARSE INBOUND SINGLE-BYTE SERIAL COMMAND TOKENS
if (Serial.available() > 0) {
    char commandToken = Serial.read();

    if (commandToken == 'F') { // Fire Request
        if (!isSolenoidActive && !isReloading) {
            digitalWrite(RELAY_SOLENOID_PIN, LOW); // Trigger active-low relay coil
            solenoidTriggerTime = currentMillis;
            isSolenoidActive = true;
        }
    }
    else if (commandToken == 'R') { // Reset System Request
        digitalWrite(RELAY_SOLENOID_PIN, HIGH);
        isSolenoidActive = false;
        isReloading = false;
        sortingServo.write(0);
    }
}

// 4. SENSOR SCAN MATRIX EVALUATIONS (Active-Low falling edge loop tracking)
for (int i = 0; i < 5; i++) {
    bool currentReading = digitalRead(IR_PINS[i]);

    if (currentReading == LOW && lastSensorState[i] == HIGH) {
        char targetHitToken = (char)('1' + i); // Formats array index dynamically to
chars '1'-'5'
        Serial.write(targetHitToken);
    }
}

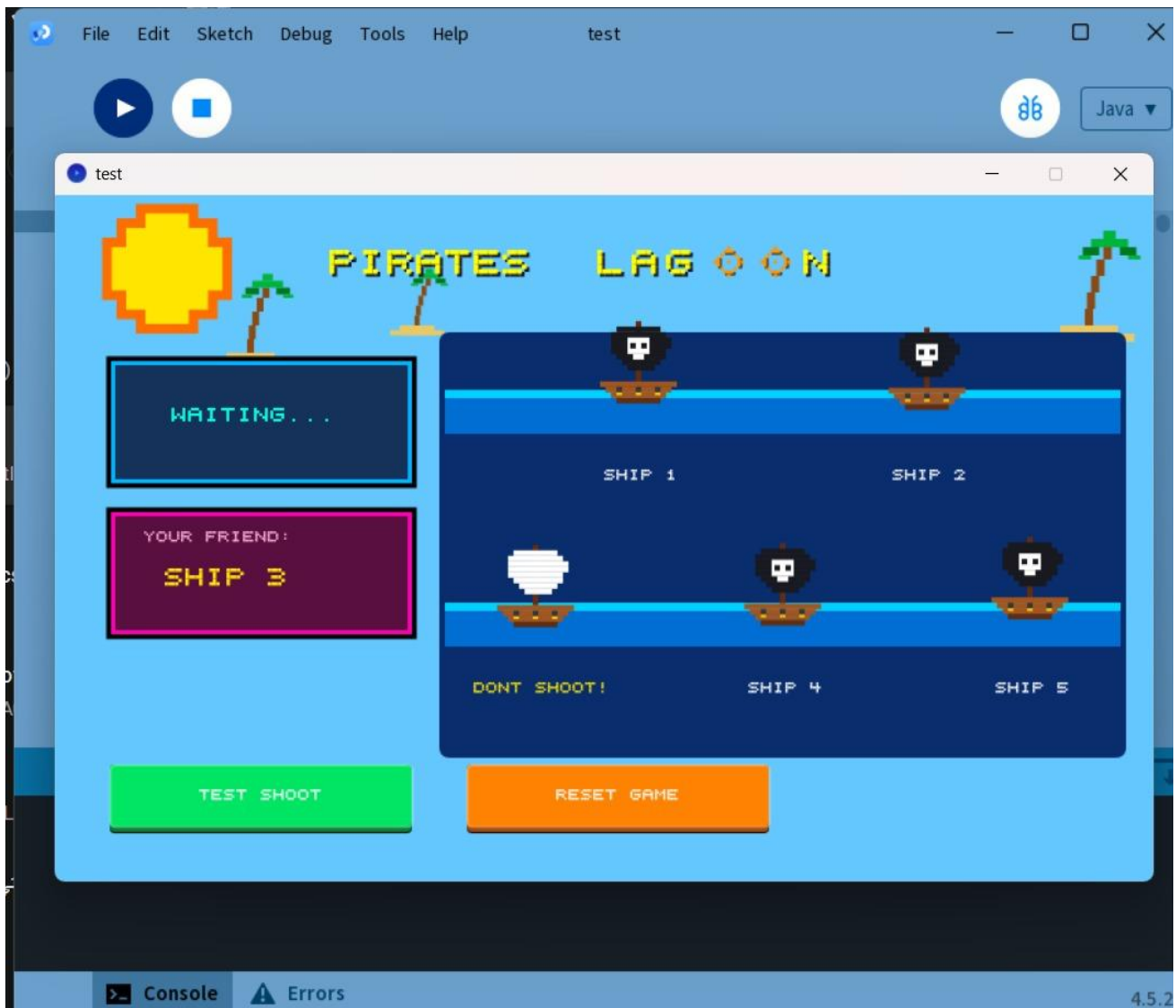
```

PIRATES LAGOON

```
    Serial.flush();  
  }  
  lastSensorState[i] = currentReading;  
}  
}
```

PIRATES LAGOON

8.2 Processing.io



CODE:

```
import processing.serial.*;

Serial bluetoothPort;

boolean shotIsFiring = false;
int firingTimer = 0;
boolean[] shipTargets = new boolean[5];
int playerAssignedShip = 1;
boolean gameOver = false;
boolean gameWon = false;
```


PIRATES LAGOON

```
// Animation variable for the floaty wave oscillation loop
float floatAngle = 0;

void setup() {
  size(800, 500);
  pixelDensity(1); // Force crisp blocky pixel alignment
  frameRate(60);

  // --- BLUETOOTH SERIAL SYSTEM INITIALIZATION ---
  // List all available serial COM ports to console window
  printArray(Serial.list());
  // Adjust index [0] below to match your HC-05 device port mapping index
  String portName = Serial.list()[0];
  bluetoothPort = new Serial(this, portName, 9600);
  bluetoothPort.bufferUntil('\n'); // Trigger read events when a newline feeds in

  clearBoard();
}

void draw() {
  // 1. LIGHTENED CARTOON SKY BACKGROUND
  background(100, 200, 255);

  // Update wave animation angle increment
  floatAngle += 0.05;

  // 2. RETRO SCENERY
  drawPixelSun(80, 52);
  drawPixelPalmTree(140, 115, 2.5);
  drawPixelPalmTree(750, 95, 3.0);
  drawPixelPalmTree(262, 95, 1.8);

  // 3. CENTERED PIXEL TITLE
  drawPixelTitle();

  // 4. DASHBOARD PANELS
  if (shotIsFiring) {
    drawPixelPanel(40, 120, 220, 90, color(255, 0, 68), color(120, 10, 40));
    drawPixelText("SHOT FIRED!", 65, 155, 2, color(255, 255, 255));
    firingTimer++;
    if (firingTimer > 12) {
      shotIsFiring = false;
      firingTimer = 0;
    }
  } else {
    drawPixelPanel(40, 120, 220, 90, color(0, 180, 255), color(20, 50, 90));
    drawPixelText("WAITING...", 85, 155, 2, color(0, 255, 221));
  }
}
```

PIRATES LAGOON

```
// Friend Ship Assign Box
drawPixelPanel(40, 230, 220, 90, color(255, 0, 180), color(90, 15, 65));
drawPixelText("YOUR FRIEND:", 65, 245, 1.5, color(255, 180, 220));
drawPixelText("SHIP " + playerAssignedShip, 80, 272, 2.5, color(255, 230, 0));

// 5. BRIGHT TROPICAL PIXEL OCEAN LAYERS
drawLayeredPixelOcean(280, 100, 500, 310);

// 6. TWO-ROW TACTICAL FLEET GRID (Pixel Art Ships)
for (int i = 0; i < 5; i++) {
    int currentShipNumber = i + 1;
    boolean isPlayerShip = (currentShipNumber == playerAssignedShip);

    float xPos, yPos;
    if (i < 2) {
        xPos = 425 + (i * 210);
        yPos = 135;
    } else {
        xPos = 350 + ((i - 2) * 180);
        yPos = 290;
    }

    if (shipTargets[i]) {
        draw8BitShip(xPos, yPos, true, isPlayerShip);
        drawPixelText("SUNK!", (int)xPos - 22, (int)yPos + 65, 1.5, color(255, 40,
40));
    } else {
        float currentFloatOffset = sin(floatAngle + i) * 5.0;
        draw8BitShip(xPos, yPos + currentFloatOffset, false, isPlayerShip);

        if (isPlayerShip) {
            drawPixelText("DONT SHOOT!", (int)xPos - 45, (int)yPos + 65, 1.5,
color(255, 230, 0));
        } else {
            drawPixelText("SHIP " + currentShipNumber, (int)xPos - 25, (int)yPos + 65,
1.5, color(255, 255, 255));
        }
    }
}

// 7. RETRO END GAME STATE OVERLAYS
if (gameOver) {
    fill(20, 15, 35, 240);
    rect(280, 110, 490, 290);
    stroke(255, 0, 68);
    strokeWeight(4);
    noFill();
}
```

PIRATES LAGOON

```

    rect(285, 115, 480, 280);

    drawPixelText("GAME OVER!", 405, 195, 3.5, color(255, 0, 68));
    drawPixelText("YOU HIT YOUR FRIEND SHIP", 345, 255, 1.5, color(255, 255, 255));
}
else if (gameWon) {
    fill(15, 35, 20, 240);
    rect(280, 110, 490, 290);
    stroke(0, 255, 100);
    strokeWeight(4);
    noFill();
    rect(285, 115, 480, 280);

    drawPixelText("VICTORY!", 420, 195, 3.5, color(0, 255, 100));
    drawPixelText("YOU SUNK ALL ENEMY SHIPS!", 335, 255, 1.5, color(255, 255,
255));
}
}

// =====
// BLUETOOTH STREAM RUNTIME INTERRUPT PARSER
// =====
void serialEvent(Serial p) {
    String incomingMessage = p.readStringUntil('\n');
    if (incomingMessage == null) return;

    incomingMessage = trim(incomingMessage);
    if (incomingMessage.length() == 0) return;

    // Pattern parsing check matching current tokens
    if (incomingMessage.startsWith("I")) {
        clearBoard();
        playerAssignedShip = int(incomingMessage.substring(1));
    }
    else if (incomingMessage.equals("FIRE")) {
        shotIsFiring = true;
        firingTimer = 0;
    }
    else if (incomingMessage.startsWith("H")) {
        int shipHitIndex = int(incomingMessage.substring(1)) - 1;
        if (shipHitIndex >= 0 && shipHitIndex < 5) {
            shipTargets[shipHitIndex] = true;
        }
    }
    else if (incomingMessage.equals("LOSE")) {
        shipTargets[playerAssignedShip - 1] = true;
        gameOver = true;
    }
}

```

PIRATES LAGOON

```

else if (incomingMessage.equals("WIN")) {
    gameWon = true;
}
}

void clearBoard() {
    gameOver = false;
    gameWon = false;
    shotIsFiring = false;
    firingTimer = 0;
    for (int i = 0; i < 5; i++) {
        shipTargets[i] = false;
    }
}

// Custom Render: Draws authentic pixel text line using hardcoded character
// matrices
void drawPixelText(String txt, int x, int y, float pixelScale, color c) {
    pushMatrix();
    translate(x, y);
    fill(c);
    noStroke();

    for (int i = 0; i < txt.length(); i++) {
        char ch = txt.charAt(i);
        int[][] glyph = getPixelGlyph(ch);

        for (int row = 0; row < 5; row++) {
            for (int col = 0; col < 5; col++) {
                if (glyph[row][col] == 1) {
                    rect(col * pixelScale, row * pixelScale, pixelScale, pixelScale);
                }
            }
        }
        translate(6 * pixelScale, 0);
    }
    popMatrix();
}

// Custom Render: Decorative double-bordered UI layout panel
void drawPixelPanel(int x, int y, int w, int h, color border, color bg) {
    fill(0);
    stroke(0);
    strokeWeight(6);
    rect(x, y, w, h);

    fill(bg);
    stroke(border);
}

```

PIRATES LAGOON

```
strokeWeight(3);
rect(x+2, y+2, w-4, h-4);
}

// Custom Render: Title layout engine with isolated letters parameters
void drawPixelTitle() {
  int startX = width / 2 - 200;
  color titleColor = color(255, 255, 0);
  color shadowColor = color(20, 20, 40);

  // --- WORD 1: PIRATES ---
  drawPixelText("PIRATES", startX + 2, 42, 3.5, shadowColor);
  drawPixelText("PIRATES", startX, 40, 3.5, titleColor);

  // --- WORD 2: LAGOON ---
  drawPixelText("L", startX + 197, 42, 3.5, shadowColor);
  drawPixelText("L", startX + 195, 40, 3.5, titleColor);
  drawPixelText("A", startX + 222, 42, 3.5, shadowColor);
  drawPixelText("A", startX + 220, 40, 3.5, titleColor);
  drawPixelText("G", startX + 247, 42, 3.5, shadowColor);
  drawPixelText("G", startX + 245, 40, 3.5, titleColor);

  drawPixelWheel(startX + 282, 40);
  drawPixelWheel(startX + 316, 40);

  drawPixelText("N", startX + 347, 42, 3.5, shadowColor);
  drawPixelText("N", startX + 345, 40, 3.5, titleColor);
}

// Custom Render: Detailed pixelated 8-spoke steering wheel
void drawPixelWheel(int x, int y) {
  pushMatrix();
  translate(x, y);

  fill(20, 20, 40);
  noStroke();
  float p = 2.5;
  rect(2*p+1, 1, 3*p, p); rect(1, 2*p+1, p, 3*p); rect(6*p+1, 2*p+1, p, 3*p);
  rect(2*p+1, 6*p+1, 3*p, p);

  fill(255, 150, 0);
  rect(2*p, 0, 3*p, p); rect(0, 2*p, p, 3*p); rect(6*p, 2*p, p, 3*p); rect(2*p,
6*p, 3*p, p);
  rect(3*p, 3*p, p, p);
  rect(3*p, -1*p, p, p); rect(3*p, 7*p, p, p); rect(-1*p, 3*p, p, p); rect(7*p,
3*p, p, p);
  rect(1*p, 1*p, p, p); rect(5*p, 1*p, p, p); rect(1*p, 5*p, p, p); rect(5*p, 5*p,
p, p);
}
```

PIRATES LAGOON

```

    popMatrix();
}

// Custom Render: Renders a beautifully defined 8-bit retro sun
void drawPixelSun(int x, int y) {
    pushMatrix();
    translate(x, y);
    noStroke();
    float p = 3.5;

    fill(255, 110, 0);
    rect(-6*p, -13*p, 13*p, 27*p);
    rect(-13*p, -6*p, 27*p, 13*p);
    rect(-10*p, -10*p, 21*p, 21*p);

    fill(255, 230, 0);
    rect(-4*p, -11*p, 9*p, 23*p);
    rect(-11*p, -4*p, 23*p, 9*p);
    rect(-8*p, -8*p, 17*p, 17*p);

    popMatrix();
}

// Custom Render: Renders a retro blocky 8-bit palm tree asset signature with
island sand base
void drawPixelPalmTree(int x, int y, float p) {
    pushMatrix();
    translate(x, y);
    noStroke();

    color sandYellow = color(235, 200, 100);
    color trunkBrown = color(140, 75, 20);
    color leafGreen = color(0, 180, 60);
    color darkLeaf = color(0, 120, 30);

    fill(sandYellow);
    rect(-6*p, 0, 14*p, 4*p);
    rect(-10*p, 2*p, 22*p, 2*p);

    fill(trunkBrown);
    rect(0, 0, 2*p, -4*p);
    rect(0.5*p, -4*p, 2*p, -4*p);
    rect(1.5*p, -8*p, 2*p, -4*p);
    rect(2.8*p, -12*p, 2*p, -4*p);
    rect(4.5*p, -16*p, 2*p, -3*p);

    translate(5.5*p, -17*p);
    fill(darkLeaf);

```

PIRATES LAGOON

```

rect(-6*p, -2*p, 4*p, 2*p);
rect(3*p, -2*p, 4*p, 2*p);

fill(leafGreen);
rect(-4*p, -4*p, 9*p, 2*p);
rect(-1*p, -6*p, 3*p, 2*p);
rect(-7*p, 0, 3*p, p);
rect(5*p, 0, 3*p, p);

popMatrix();
}

// Custom Render: Bright, flooded tropical retro ocean box environment
void drawLayeredPixelOcean(int x, int y, int w, int h) {
    noStroke();

    color deepAbyss = color(10, 45, 110);
    color midWater = color(0, 110, 210);
    color surfCyan = color(0, 210, 255);

    fill(deepAbyss);
    rect(x, y, w, h, 8);

    fill(midWater);
    rect(x + 4, y + 42, w - 8, 32);
    fill(surfCyan);
    rect(x + 4, y + 42, w - 8, 6);

    fill(midWater);
    rect(x + 4, y + 197, w - 8, 32);
    fill(surfCyan);
    rect(x + 4, y + 197, w - 8, 6);
}

// Advanced 8-Bit Ship Engine: Splitting hull physics with frame-animated
// smoke/fire particles
void draw8BitShip(float x, float y, boolean isDestroyed, boolean isPlayerShip) {
    pushMatrix();
    translate(x, y);
    noStroke();

    float p = 4.0;

    color woodSkin = color(165, 85, 30);
    color woodTrim = color(100, 45, 15);
    color gunMetal = color(50, 55, 65);
    color neonGold = color(255, 200, 0);
    color maskBlack = color(25, 25, 30);

```

PIRATES LAGOON

```
color boneWhite = color(255, 255, 255);

if (isDestroyed) {
  pushMatrix();
  translate(-16, 26);
  rotate(radians(-40));
  fill(woodSkin);
  rect(-6*p, 0, 6*p, 3*p);
  fill(woodTrim);
  rect(-7*p, -p, 7*p, p);
  popMatrix();

  pushMatrix();
  translate(18, 18);
  rotate(radians(48));
  fill(woodTrim);
  rect(-0.5*p, -4*p, p, 5*p);
  fill(woodSkin);
  rect(0, 0, 6*p, 3*p);
  fill(woodTrim);
  rect(0, -p, 7*p, p);
  popMatrix();

  int frameCheck = frameCount % 30;
  if (frameCheck < 15) {
    fill(255, 50, 0);
    rect(-2*p, -1*p, 2*p, 2*p);
    rect(1*p, -3*p, p, p);
    fill(255, 230, 0);
    rect(-1*p, -1*p, p, p);
  } else {
    fill(255, 150, 0);
    rect(-1*p, -2*p, 3*p, 1.5*p);
    rect(-3*p, 0, p, p);
    fill(255, 255, 255);
    rect(0, -1*p, p, p);
  }

  fill(60, 65, 75, 200);
  rect(-4*p, -4*p - (frameCheck/3.0), 2*p, 2*p);
  rect(2*p, -7*p - (frameCheck/4.0), 3*p, 2*p);
  rect(0, -10*p - (frameCheck/5.0), 2*p, 2*p);
} else {
  fill(woodTrim);
  rect(-0.5*p, -10*p, p, 11*p);

  if (isPlayerShip) {
```


PIRATES LAGOON

```

        fill(boneWhite);
    } else {
        fill(maskBlack);
    }
    rect(-4*p, -9*p, 8*p, p);
    rect(-5*p, -8*p, 10*p, p);
    rect(-5*p, -7*p, 11*p, p);
    rect(-5*p, -6*p, 11*p, p);
    rect(-5*p, -5*p, 10*p, p);
    rect(-4*p, -4*p, 9*p, p);
    rect(-3*p, -3*p, 7*p, p);
    rect(-2*p, -2*p, 5*p, p);

    if (!isPlayerShip) {
        fill(boneWhite);
        rect(-2*p, -7*p, 4*p, 3*p);
        rect(-1*p, -4*p, 2*p, p);
        fill(maskBlack);
        rect(-1.5*p, -6*p, p, p); rect(0.5*p, -6*p, p, p);
    }

    fill(woodSkin);
    rect(-7*p, 1*p, 14*p, p);
    rect(-6*p, 2*p, 12*p, p);
    rect(-5*p, 3*p, 10*p, p);
    fill(woodTrim);
    rect(-4*p, 4*p, 8*p, p);

    fill(gunMetal);
    rect(-4*p, 2*p, p, p); rect(-0.5*p, 2*p, p, p); rect(3*p, 2*p, p, p);
    fill(neonGold);
    rect(-4*p, 3*p, p, 0.5*p); rect(-0.5*p, 3*p, p, 0.5*p); rect(3*p, 3*p, p,
0.5*p);
    }
    popMatrix();
}

// Low-Level Library Font Mapping Matrix
int[][] getPixelGlyph(char c) {
    int[][] g = new int[5][5];
    switch(Character.toUpperCase(c)) {
        case 'P': return new
int[][]{{1,1,1,1,0},{1,0,0,0,1},{1,1,1,1,0},{1,0,0,0,0},{1,0,0,0,0}};
        case 'I': return new
int[][]{{0,1,1,1,0},{0,0,1,0,0},{0,0,1,0,0},{0,0,1,0,0},{0,1,1,1,0}};
        case 'R': return new
int[][]{{1,1,1,1,0},{1,0,0,0,1},{1,1,1,1,0},{1,0,0,1,0},{1,0,0,0,1}};

```

PIRATES LAGOON

```

    case 'A': return new
int[][]{{0,1,1,1,0},{1,0,0,0,1},{1,1,1,1,1},{1,0,0,0,1},{1,0,0,0,1}};
    case 'T': return new
int[][]{{1,1,1,1,1},{0,0,1,0,0},{0,0,1,0,0},{0,0,1,0,0},{0,0,1,0,0}};
    case 'E': return new
int[][]{{1,1,1,1,1},{1,0,0,0,0},{1,1,1,1,0},{1,0,0,0,0},{1,1,1,1,1}};
    case 'S': return new
int[][]{{0,1,1,1,1},{1,0,0,0,0},{0,1,1,1,0},{0,0,0,0,1},{1,1,1,1,0}};
    case 'L': return new
int[][]{{1,0,0,0,0},{1,0,0,0,0},{1,0,0,0,0},{1,0,0,0,0},{1,1,1,1,1}};
    case 'G': return new
int[][]{{0,1,1,1,1},{1,0,0,0,0},{1,0,1,1,1},{1,0,0,0,1},{0,1,1,1,0}};
    case 'N': return new
int[][]{{1,0,0,0,1},{1,1,0,0,1},{1,0,1,0,1},{1,0,0,1,1},{1,0,0,0,1}};
    case 'W': return new
int[][]{{1,0,0,0,1},{1,0,0,0,1},{1,0,1,0,1},{1,1,0,1,1},{1,0,0,0,1}};
    case 'O': return new
int[][]{{0,1,1,1,0},{1,0,0,0,1},{1,0,0,0,1},{1,0,0,0,1},{0,1,1,1,0}};
    case 'H': return new
int[][]{{1,0,0,0,1},{1,0,0,0,1},{1,1,1,1,1},{1,0,0,0,1},{1,0,0,0,1}};
    case 'U': return new
int[][]{{1,0,0,0,1},{1,0,0,0,1},{1,0,0,0,1},{1,0,0,0,1},{0,1,1,1,0}};
    case 'M': return new
int[][]{{1,0,0,0,1},{1,1,0,1,1},{1,0,1,0,1},{1,0,0,0,1},{1,0,0,0,1}};
    case 'V': return new
int[][]{{1,0,0,0,1},{1,0,0,0,1},{1,0,0,0,1},{0,1,0,1,0},{0,0,1,0,0}};
    case 'Y': return new
int[][]{{1,0,0,0,1},{0,1,0,1,0},{0,0,1,0,0},{0,0,1,0,0},{0,0,1,0,0}};
    case 'D': return new
int[][]{{1,1,1,1,0},{1,0,0,0,1},{1,0,0,0,1},{1,0,0,0,1},{1,1,1,1,0}};
    case 'K': return new
int[][]{{1,0,0,0,1},{1,0,0,1,0},{1,1,1,0,0},{1,0,0,1,0},{1,0,0,0,1}};
    case 'F': return new
int[][]{{1,1,1,1,1},{1,0,0,0,0},{1,1,1,1,0},{1,0,0,0,0},{1,0,0,0,0}};
    case '1': return new
int[][]{{0,0,1,0,0},{0,1,1,0,0},{0,0,1,0,0},{0,0,1,0,0},{0,1,1,1,0}};
    case '2': return new
int[][]{{0,1,1,1,0},{1,0,0,0,1},{0,0,0,1,0},{0,0,1,0,0},{1,1,1,1,1}};
    case '3': return new
int[][]{{1,1,1,1,0},{0,0,0,0,1},{0,1,1,1,0},{0,0,0,0,1},{1,1,1,1,0}};
    case '4': return new
int[][]{{1,0,0,1,0},{1,0,0,1,0},{1,1,1,1,1},{0,0,0,1,0},{0,0,0,1,0}};
    case '5': return new
int[][]{{1,1,1,1,1},{1,0,0,0,0},{1,1,1,1,0},{0,0,0,0,1},{1,1,1,1,0}};
    case ':': return new
int[][]{{0,0,0,0,0},{0,0,1,0,0},{0,0,0,0,0},{0,0,1,0,0},{0,0,0,0,0}};
    case '.': return new
int[][]{{0,0,0,0,0},{0,0,0,0,0},{0,0,0,0,0},{0,0,0,0,0},{0,0,1,0,0}};

```

PIRATES LAGOON

```
    case '!': return new  
int[][]{{0,0,1,0,0},{0,0,1,0,0},{0,0,1,0,0},{0,0,0,0,0},{0,0,1,0,0}};  
    }  
    return g;  
}
```

9. Safety Considerations

Safety was a paramount concern throughout the design process. The following safety measures were implemented:

- **All wiring is completely hidden and inaccessible to users**
- **No sharp edges - all corners have minimum 3mm radius**
- **No small parts that could pose choking hazard**
- **Stable base design prevents tipping**
- **Electrical safety: reverse polarity protection**